

SECURE AGENTIC AI BANKING DEMO

Architecture Decision Rationale & Review Defense Guide

Why the OCI Always Free runbook is built the way it is

Purpose: make every major runbook decision explainable - technically, financially, operationally, and from a risk/governance perspective.

CORE THESIS

Agentic AI can increase employee capacity without removing human accountability.

Use this guide when someone asks: "Why did you do it that way?"

1. Executive Summary

The runbook optimizes for a credible, explainable proof of concept (POC) - not for pretending a free demo is a production banking platform.

The design intentionally proves four things at the same time:

- Business value: an employee can resolve a familiar card-decline problem with less system hunting and policy lookup.
- Agentic behavior: the system gathers evidence, retrieves policy, reasons over the evidence, recommends an action, pauses for approval, then executes a controlled mock action.
- Security and governance: authorization is applied before sensitive information reaches the Large Language Model (LLM), consequential actions require human approval, and material steps are auditable.
- Cost discipline: Oracle Cloud Infrastructure (OCI) Always Free resources and a local open model keep the infrastructure demonstration free while preserving an easy path to a paid enterprise model later.

The 30-second architecture defense

“I deliberately kept the POC small and transparent. Oracle AI Database 26ai holds both structured banking data and vectorized policy content. Python orchestrates explicit tools. A local Ollama model performs the reasoning so the demo does not depend on a paid model API. Authorization is enforced in the data/tool layer before the model sees context. The AI proposes the next action, a human approves consequential changes, a mock API performs the action, and Oracle records the audit trail. That lets the demo prove agentic workflow, RAG, access control, human accountability, and auditability without over-engineering the platform.”

What this is - and what it is not

This POC IS	This POC IS NOT
A working demonstration of the employee-assist workflow	A claim that the free topology satisfies every production banking control
A governed agent with explicit tools and approval gates	An autonomous AI with unrestricted access to banking systems
A demonstration using synthetic customer and transaction data	A test using real customer, card, fraud, or regulated banking data
A reusable architecture pattern	A final production landing zone or disaster recovery design
A model-portable implementation	A dependency on one commercial LLM provider

2. Design Principles Used to Make the Runbook Decisions

P1 - Prove the business outcome first	Every component must help demonstrate faster, more consistent employee resolution of a customer problem.
P2 - Keep consequential decisions human-owned	The agent can investigate and recommend; it cannot silently make account-changing decisions.
P3 - Enforce access before generation	The model only receives authorized context. “Do not reveal this” prompts are not treated as an authorization system.
P4 - Make agent behavior observable	Tool calls, retrieved policy, recommendations, approvals, and actions are visible and auditable.
P5 - Use the fewest moving parts that prove the thesis	The POC avoids infrastructure whose only purpose would be to look enterprise-scale.
P6 - Preserve an enterprise upgrade path	Free-tier choices should be replaceable rather than architectural dead ends.
P7 - Separate POC controls from production controls	The POC implements application-level governance and documents what would be strengthened for production.
P8 - Avoid real regulated data	Synthetic data lets us demonstrate banking behavior without creating unnecessary privacy or compliance exposure.

Decision hierarchy

Business problem -> Governance model -> Data boundaries -> Agent workflow -> Technology choices -> User interface

Why this order matters: we did not begin with “Which LLM should we use?” We first decided what the employee needs, what AI is allowed to do, and where human accountability must remain. The model is therefore a replaceable implementation detail rather than the architecture.

3. Decision Snapshot

ID	Decision	Primary reason
01	Card-decline resolution	Familiar, measurable, easy to demonstrate
02	Employee-assist, not employee replacement	Human accountability and adoption story
03	Synthetic banking data	Zero need for real regulated data
04	OCI Always Free	Prove value before spending
05	Autonomous AI Database 26ai	Relational + vector in one governed data platform
06	Ampere A1 VM	Free application/model host with enough memory for a small model
07	Python orchestration	Transparent, inspectable agent logic
08	No LangChain initially	Avoid abstraction hiding the controls we need to teach

09	Ollama + small local model	No paid model API required for the POC
10	Nomic 768-dimension embeddings	Small, local, stable dimensional contract
11	Cosine vector search	Simple semantic retrieval appropriate to text embeddings
12	Role filtering before retrieval/generation	Prevent restricted data from reaching the LLM
13	mTLS wallet for ADB connectivity	Secure encrypted connection with official Oracle path
14	Streamlit behind SSH tunnel	Simple demo UI without opening an app port publicly
15	Mock banking action	Prove action-taking without touching a real banking service
16	Human approval gate	AI proposes; employee authorizes consequential action
17	Audit log in Oracle	Trace what happened and who approved it
18	OpenAI as later swappable endpoint	Enterprise option without making the free POC dependent on paid API usage

4. Detailed Architecture Decision Records

ADR-01 - Use a declined-card transaction as the business scenario

Decision	Use a familiar card-decline resolution workflow as the transaction dilemma.
Why	Almost anyone understands “Why was my card declined?” immediately. The scenario naturally spans transaction data, account status, fraud signals, customer context, policy lookup, customer communication, and an action that should remain controlled.
Alternative considered	International wire investigation, generic policy chatbot, IT incident response.
Trade-off accepted	A card decline is less operationally complex than a wire investigation, but that simplicity improves executive comprehension and keeps the demo focused on the AI pattern rather than banking jargon.
How to defend it	“I chose a high-frequency, relatable customer problem that still contains the same enterprise challenges: fragmented data, policy, authorization, judgment, and action.”

ADR-02 - Position AI as employee augmentation

Decision	The agent assists the employee; it does not remove the employee from the consequential decision.
Why	The POC is intended to demonstrate capacity gain while preserving accountability. That aligns the technical workflow with the executive message and makes the control model visible.
Alternative considered	Fully autonomous transaction resolution; simple non-agentic chatbot.
Trade-off accepted	Some potential automation is deliberately left on the table so that the demo can show a clear governance boundary.
How to defend it	“AI handles evidence gathering and coordination. The employee owns the decision and customer relationship.”

ADR-03 - Use synthetic banking data only

Decision	Create fictional customers, accounts, transactions, fraud events, and policies.
Why	The demo does not need real data to prove retrieval, authorization, reasoning, human approval, or auditability. Synthetic data reduces privacy, security, legal, and handling complexity.

Alternative considered	Masked production extracts; public banking datasets; manually anonymized real transactions.
Trade-off accepted	Synthetic data is less realistic than production data, but controllability is an advantage for a repeatable demo because we know exactly what the correct answer should be.
How to defend it	“There is no business value in introducing real regulated data into a POC whose objective is architectural proof.”

ADR-04 - Use OCI Always Free as the infrastructure constraint

Decision	Build the POC using Always Free Autonomous AI Database and Ampere A1 Compute where available.
Why	Cost is part of the architecture exercise: prove that the workflow and governance pattern work before paying for scale, resilience, or managed AI services.
Alternative considered	Paid Autonomous Database; OCI Generative AI; Kubernetes; GPU compute.
Trade-off accepted	The free tier constrains compute, networking options, and production-grade features. Those constraints are acceptable because the target is a POC, not a production service.
How to defend it	“We separated proof of value from production scale. I did not want infrastructure cost to be confused with whether the design works.”
Evidence basis	Oracle documents Always Free resources for small-scale applications and proof-of-concept testing; A1 free allocation is equivalent to 2 OCPUs and 12 GB memory. [S1][S2]
Boundary / caution	Always Free capacity and database-version availability are region-dependent. Verify the Console in the tenancy home region before building.

ADR-05 - Use Oracle Autonomous AI Database 26ai

Decision	Keep relational banking data, policy chunks, embeddings, and audit records in the same Oracle database.
Why	The banking evidence is naturally relational while policy retrieval is semantic. Oracle AI Database 26ai lets one governed platform support both patterns and avoids adding a separate vector database solely for the demo.
Alternative considered	PostgreSQL + pgvector; standalone vector database; local files + FAISS; separate policy store.

Trade-off accepted	The POC becomes Oracle-centric, but that is intentional because the portfolio also demonstrates modern Oracle AI capabilities.
How to defend it	“The point was to show that existing enterprise data and AI retrieval can coexist without creating another data silo.”
Evidence basis	Oracle 26ai documents native VECTOR support and vector-distance functions; Always Free supports Oracle AI Database 26ai in supported regions. [S1][S3][S4]

ADR-06 - Use an Ampere A1 VM for the app and local model

Decision	Host Python, Streamlit, Ollama, and the local model on VM.Standard.A1.Flex within the free allocation.
Why	One VM keeps orchestration, UI, and model runtime easy to understand while making use of the free A1 allocation. ARM64 is supported by Ollama.
Alternative considered	Separate application and model VMs; x86 VM; managed container platform; GPU instance.
Trade-off accepted	CPU inference is slower and the model must remain small. The POC prioritizes deterministic demonstration over throughput.
How to defend it	“The A1 instance is intentionally modest. If the workflow works under constrained compute, model hosting can be upgraded independently later.”
Evidence basis	OCI documents the Always Free A1 allocation; Ollama publishes an ARM64 Linux package. [S2][S6]

ADR-07 - Use Python as the orchestration layer

Decision	Implement explicit tools and workflow transitions in plain Python.
Why	Python has first-class Oracle and Ollama clients, is readable in a demo, and makes each governance boundary inspectable. It also keeps the POC close to the actual control flow.
Alternative considered	Java; JavaScript/TypeScript; PL/SQL-only orchestration; low-code agent platform.
Trade-off accepted	Python is not the only enterprise implementation choice, but portability is more valuable than language standardization at the POC stage.
How to defend it	“I optimized for transparency. Every query, retrieval, decision boundary, approval, and action can be shown directly in code.”

Evidence basis	Oracle documents Python connectivity to Autonomous AI Database and Python support for vector applications. [S5] [S4]
-----------------------	--

ADR-08 - Do not use LangChain in the first version

Decision	Use a small explicit state machine/functions rather than a general agent framework.
Why	The purpose of the POC is to teach and defend the control architecture. Framework abstractions can make it harder to show which code authorizes data, chooses tools, pauses for approval, and records audit events.
Alternative considered	LangChain; LlamaIndex; a managed agent framework.
Trade-off accepted	We write a little more glue code and give up some convenience. That is acceptable because the codebase is deliberately small.
How to defend it	“I wanted the agent mechanics visible. Once the behavior is proven, a framework can replace plumbing without changing the control model.”
Evidence basis	Oracle provides a LangChain integration, so adoption later remains available; it is deferred, not rejected. [S7]

ADR-09 - Use Ollama and Qwen2.5 1.5B for the free POC

Decision	Run a small local instruction model with Ollama instead of requiring a paid model API.
Why	The free A1 VM has limited memory. Qwen2.5 1.5B is small enough to fit comfortably and is sufficient for controlled synthesis over short, grounded context. Ollama keeps the model endpoint local and swappable.
Alternative considered	OpenAI API; OCI Generative AI; larger local model; GPU-hosted model.
Trade-off accepted	Answer quality and latency will not match a frontier enterprise model. That is acceptable because the POC tests orchestration and governance rather than benchmark leadership.
How to defend it	“I separated model quality from architecture. The POC proves the workflow with a zero-API-cost model, and the model adapter can be upgraded later.”
Evidence basis	Ollama lists qwen2.5:1.5b at about 986 MB and provides ARM64 Linux support. [S6][S8]

ADR-10 - Use Nomic Embed Text with a 768-dimension vector column

Decision	Generate policy embeddings locally with nomic-embed-text and store them in VECTOR(768, FLOAT32).
Why	A local embedding model avoids an external embedding API, makes ingestion reproducible, and provides a fixed dimensional contract. The model metadata reports an embedding length of 768.
Alternative considered	OpenAI embeddings; Oracle in-database ONNX embeddings; sentence-transformer model; different vector dimension.
Trade-off accepted	Changing embedding models later may require re-embedding policy content and possibly changing the vector column dimension.
How to defend it	“The embedding model is a data-contract decision. I chose a small local model with a known 768-dimensional output so the entire retrieval pipeline stays free and reproducible.”
Evidence basis	Ollama model metadata for nomic-embed-text reports a 768 embedding length. [S9][S10]

ADR-11 - Use cosine distance for semantic policy retrieval

Decision	Rank policy chunks using VECTOR_DISTANCE(..., COSINE).
Why	Cosine distance is a straightforward, supported semantic-similarity metric for text embeddings. For a small POC corpus, exact ranking keeps behavior easy to explain and debug.
Alternative considered	Euclidean distance; inner product; approximate vector index from day one.
Trade-off accepted	Exact search will not be the scaling answer for a very large corpus, but the POC corpus is tiny and correctness/observability matter more than optimization.
How to defend it	“I optimized first for retrieval correctness and explainability. Indexing is a scaling optimization we can add after we establish a baseline.”
Evidence basis	Oracle documents COSINE as a VECTOR_DISTANCE metric and as the default in common cases. [S3]

ADR-12 - Apply role authorization before vector retrieval and before the LLM

Decision	Filter policy rows by the employee role and expose only role-appropriate fraud fields before generation.
-----------------	--

Why	The model should never become the authorization engine. If restricted data is excluded from retrieval/tool output, the model cannot accidentally disclose what it never received.
Alternative considered	Retrieve everything and tell the model not to disclose restricted data; post-filter generated answers only.
Trade-off accepted	The application/data layer must maintain authorization rules. That is extra engineering, but it is the correct trust boundary.
How to defend it	“Prompt instructions are not access control. Authorization belongs in deterministic systems before the model sees context.”

ADR-13 - Use an Autonomous Database wallet and mutual TLS (mTLS)

Decision	Connect python-oracledb to Autonomous AI Database using the downloaded wallet and mTLS.
Why	This follows an official Oracle connection path, provides encrypted transport and client credentials, and avoids needing Oracle Client libraries when python-oracledb Thin mode is used.
Alternative considered	One-way TLS without a wallet; Thick mode with Instant Client; opening unsecured database access.
Trade-off accepted	Wallet distribution and secret protection must be handled carefully. For the POC, one application VM keeps that scope contained.
How to defend it	“I chose the secure documented path with the smallest client footprint: python-oracledb Thin mode plus the Autonomous wallet.”
Evidence basis	Oracle and python-oracledb documentation describe mTLS wallet connectivity and Thin mode without Oracle Client libraries. [S5][S11]

ADR-14 - Use Streamlit but keep it behind an SSH tunnel

Decision	Bind Streamlit to 127.0.0.1 on the VM and reach it through SSH port forwarding.
Why	Streamlit creates a usable demo UI quickly. Binding it to localhost and tunneling avoids exposing the development app directly to the public internet or adding a load balancer/API gateway just for a POC.
Alternative considered	Open port 8501 publicly; Nginx + TLS; OCI Load Balancer; full React application.

Trade-off accepted	The presenter must establish an SSH tunnel before the demo. That minor inconvenience buys a much cleaner exposure story.
How to defend it	“I needed a UI, not a public web product. The SSH tunnel gives us the demo experience without pretending Streamlit is the production ingress.”

ADR-15 - Use a mock banking action instead of a real banking API

Decision	Represent the final action as a controlled database state change or simulated API endpoint.
Why	The POC needs to prove that the agent can move from recommendation to approved action. It does not need real card-network, fraud-platform, or core-banking connectivity to prove that control flow.
Alternative considered	Integrate a real payment sandbox; connect to a vendor demo API; stop at recommendation only.
Trade-off accepted	A mock action proves orchestration but not vendor-specific integration complexity.
How to defend it	“The architectural question is whether action is gated and auditable. Real payment connectivity would add risk and noise without changing that answer.”

ADR-16 - Require human approval before consequential action

Decision	The employee must explicitly approve or reject the proposed account/transaction change.
Why	This is the central accountability boundary in the demo. It lets the agent increase capacity while preserving human ownership for consequential customer-impacting action.
Alternative considered	Fully autonomous execution; no execution capability at all.
Trade-off accepted	Human approval can reduce theoretical automation speed, but that is a deliberate control for higher-impact actions.
How to defend it	“The AI can investigate at machine speed, but the person remains accountable for the consequential decision.”

ADR-17 - Store audit events in Oracle

Decision	Persist goal, evidence gathering, policy retrieval, recommendation, approval/rejection, and execution events.
-----------------	---

Why	A regulated-enterprise story is incomplete if the organization cannot reconstruct what the AI did and which human approved an action. Storing the trail next to the POC data also keeps the demonstration simple.
Alternative considered	Console logs only; local JSON file; external Security Information and Event Management (SIEM) platform.
Trade-off accepted	A production deployment would normally forward/centralize audit telemetry. The POC intentionally proves the event model first.
How to defend it	“An agent is easier to govern when its important decisions and actions are observable after the fact.”

ADR-18 - Keep OpenAI as a swappable enterprise model endpoint, not a free-tier dependency

Decision	Design the model call behind an adapter so OpenAI can replace the local model later without changing retrieval, authorization, approval, or audit architecture.
Why	The enterprise version may benefit from a more capable hosted model, but the OpenAI API is separately billed. Keeping it out of the free POC preserves the “no paid model API required” claim and proves model portability.
Alternative considered	Make OpenAI mandatory from day one; never support hosted models.
Trade-off accepted	The local POC will not demonstrate provider-specific OpenAI controls until that endpoint is added.
How to defend it	“The governance architecture is model-independent. A model upgrade should be a controlled substitution, not a redesign.”
Evidence basis	OpenAI documents API pricing separately from ChatGPT subscriptions and provides pay-as-you-go API pricing. [S12] [S13]

5. Why the Security Controls Are Ordered This Way

The most important security decision in the runbook is not the prompt. It is the order of operations.

Correct trust sequence

Employee identity -> deterministic authorization -> permitted tool/data retrieval -> minimum context -> LLM reasoning -> policy/approval gate -> action -> audit

Why not let the LLM decide access?

- LLM outputs are probabilistic; authorization rules need deterministic enforcement.
- If restricted data is placed into the prompt, a later prompt-injection or instruction-following failure can become a disclosure problem.
- Pre-retrieval filtering supports least-privilege and data-minimization principles.
- It creates an architecture that remains safe even when the model is swapped for a different provider/model.

Why the fraud tool returns different fields by role

CUSTOMER_SERVICE -> rule_code + safe_description
FRAUD_ANALYST -> rule_code + safe_description + fraud_score + internal_notes

Defense: the LLM is downstream of the authorization boundary. A Customer Service Representative (CSR) cannot cause the model to reveal an internal fraud score because that field was never returned to the model.

Why the agent stops before execution

- Investigation and recommendation are reversible information tasks; changing customer/account state is consequential.
- The approval gate creates a clear separation of duties between machine coordination and human accountability.
- The action is executed only after an explicit user event, which can be recorded in the audit log.

6. Defending the Seven Agentic Steps

Step	Architectural purpose	Defense line
1. Employee sets goal	The human defines intent; the AI does not invent a business objective.	“Goal ownership stays with the employee.”
2. Agent gathers evidence	Tool calls replace manual system hopping while preserving source separation.	“The agent is coordinating systems, not guessing.”
3. Agent retrieves policy	RAG grounds the answer in approved procedure instead of model memory.	“Policy is retrieved as evidence, not memorized as truth.”
4. Agent reasons across evidence	The LLM synthesizes structured facts and authorized policy into a useful explanation.	“Reasoning is bounded by supplied evidence.”
5. Agent recommends next step	The model translates evidence into a proposed action and disclosure guidance.	“Recommendation is assistance, not authorization.”
6. Human approval gate	A consequential change requires explicit human approval.	“The employee owns the decision.”
7. Agent executes + audits	The system performs only the approved mock action and records what happened.	“Action is controlled, attributable, and reviewable.”

7. Alternatives We Intentionally Deferred

OpenAI API in the first build	Adds model quality but breaks the “no paid model API required” POC constraint. Keep as a model adapter swap after the free workflow works.
OCI Generative AI	Useful enterprise option, but unnecessary to prove the workflow and may introduce cost/availability considerations.
LangChain/LlamaIndex	Excellent accelerators after the control flow is understood. Deferred to keep authorization, retrieval, approval, and audit mechanics visible.
Vector index immediately	The corpus is tiny. Exact cosine search gives a clear correctness baseline; indexing becomes meaningful when scale justifies it.
Private Autonomous endpoint	A stronger production network control but outside the Always Free topology used in this POC.
API Gateway / Load Balancer / WAF	Production ingress controls add moving parts without changing the POC thesis. Streamlit stays local behind SSH.
Kubernetes	No orchestration problem exists at POC scale. A single VM is easier to operate and explain.
Real banking API	Not needed to demonstrate an approved action. Mock execution avoids external risk and credentials.
Real customer data	No additional proof value; significant additional data-handling risk.
Fully autonomous action	Conflicts with the human-accountability thesis for consequential banking actions.

8. POC Boundaries and Production Upgrade Path

A strong architecture defense includes what you deliberately did NOT claim.

Network isolation	POC: public Autonomous endpoint with secure connection controls. Production: evaluate private endpoint/VCN patterns and enterprise ingress controls.
Identity	POC: environment-configured employee identity/role. Production: integrate enterprise Identity and Access Management (IAM), Single Sign-On (SSO), strong authentication, and mapped authorization claims.
Secrets	POC: protected local environment file/wallet. Production: managed secrets and key lifecycle.

Model	POC: small local model. Production: select hosted/local model based on quality, latency, data controls, residency, retention, support, and cost.
Availability	POC: single application VM. Production: multi-instance/high-availability design, resilience testing, disaster recovery, and operational Service Level Objectives (SLOs).
Observability	POC: Oracle audit table and console visibility. Production: centralized logs/metrics/traces and Security Information and Event Management (SIEM) integration.
Action APIs	POC: simulated state change. Production: scoped service identities, API authorization, idempotency, validation, transaction controls, rollback/compensation, and rate limits.
Evaluation	POC: known synthetic truth cases. Production: formal retrieval/generation evaluation, policy compliance testing, red-team/adversarial testing, drift monitoring, and change management.

9. Architecture Review / Interview Q&A

Q: Why Oracle instead of a dedicated vector database?

A: The demo already needs relational banking data, authorization metadata, and audit state. Oracle 26ai can store those plus vectors, which proves a converged enterprise pattern and removes an unnecessary data store from the POC.

Q: Why use vector search if the transaction itself is relational?

A: We do not vectorize the transaction. We use SQL for deterministic structured facts and vector search for unstructured policies/procedures. The point is to combine the right retrieval method for each type of data.

Q: Why not just ask the LLM to interpret the decline code?

A: Because model memory is not the bank's source of truth. We retrieve actual transaction evidence and approved policy, then ask the model to synthesize only that authorized context.

Q: Why is this agentic and not just RAG?

A: RAG is one capability inside the workflow. The agent also pursues a goal, invokes multiple tools, gathers evidence, chooses/retrieves policy context, forms a recommendation, pauses for approval, executes an allowed action, and records the result.

Q: Why is there still a human if the AI can determine the next step?

A: Because increasing employee capacity is different from transferring accountability. The demo intentionally separates rapid machine investigation from human authorization of consequential actions.

Q: What prevents the model from exposing an internal fraud score?

A: The Customer Service tool never returns that field, and restricted policy rows are filtered by role before vector retrieval. The model cannot disclose information it never receives.

Q: What happens if the model hallucinates?

A: The prompt constrains generation to supplied evidence, the UI shows the retrieved evidence/policies, and consequential action remains behind approval. Production would add stronger automated evaluation and validation gates.

Q: Why not use a bigger model?

A: The POC is constrained by free compute and is designed to prove orchestration/governance, not maximize model benchmark quality. A model adapter keeps the architecture ready for a stronger hosted model later.

Q: Why not use OpenAI now?

A: We can. We intentionally kept the base POC independent of a paid API so the full workflow can be reproduced for free. OpenAI is an enterprise model option, not an architectural dependency.

Q: Is the free-tier architecture production-ready for a bank?

A: No, and the POC does not claim that. It proves application-level workflow and governance concepts. Production would strengthen networking, identity, secrets, resilience, observability, action integration, model governance, and formal compliance controls.

Q: Why not automate the final action too?

A: For low-risk actions a bank could progressively increase autonomy. This demo chooses a human gate because it cleanly demonstrates how agentic AI can add capacity without eliminating accountability.

Q: How would you measure business value?

A: Start with average handle time, time spent navigating systems, first-contact resolution, escalation rate, policy adherence, rework, customer satisfaction, and employee experience. Compare the current workflow with the agent-assisted workflow.

10. When We Should Change These Decisions

Move from Qwen 1.5B	When grounded answer quality is insufficient or latency/quality targets justify a stronger model.
---------------------	---

Add a vector index	When policy corpus size/query volume makes exact vector search materially slow.
Introduce LangChain/LlamaIndex	When the workflow expands enough that framework-managed tools, memory, tracing, or composition reduce complexity more than they obscure controls.
Move Streamlit behind production ingress	When the app becomes multi-user, externally accessible, persistent, or requires enterprise authentication.
Replace environment identity	Before any real multi-user or regulated use; integrate enterprise IAM/SSO.
Replace mock action	When a sandbox or real service contract exists and we can apply proper API authorization, validation, idempotency, and rollback controls.
Move to paid OCI resources	When production network isolation, higher availability, greater compute, scale, or supported enterprise operations are requirements.
Use OpenAI or another hosted model	When enterprise requirements justify higher model quality/capability and the organization has approved provider, retention, residency, security, and cost controls.

11. One-Page Defense Cheat Sheet

If you remember only six points, remember these.

1	We started with the business problem, not the model. A relatable card-decline scenario shows fragmented information, policy, customer pressure, and the need for judgment.
2	We chose augmentation over replacement. The agent removes manual hunting; the employee retains customer communication and approval authority.
3	We enforce access before the LLM. Deterministic tools/SQL decide what the employee may retrieve. The model never receives restricted context for unauthorized roles.
4	We use the right retrieval for the right data. SQL retrieves transaction/account facts; vector search retrieves unstructured policy; the LLM synthesizes both.
5	We keep the free POC simple and model-portable. OCI Always Free + Python + Ollama proves the pattern without requiring a paid model API or unnecessary platform services.
6	We prove action with accountability. The agent recommends, the employee approves, a mock action executes, and the full sequence is audited.

Closing line: “The architecture is deliberately conservative about authority and deliberately aggressive about removing employee friction.”

12. Source Anchors for Technical Decisions

These are official/current technical references supporting the platform-specific choices in the companion guide. Product availability and pricing can change; re-verify before a future rebuild.

S1 - Oracle - Always Free Autonomous AI Database: <https://docs.oracle.com/en-us/iaas/autonomous-database-serverless/doc/autonomous-always-free.html>

S2 - Oracle - Always Free Resources (OCI Compute and database allocations): https://docs.oracle.com/en-us/iaas/Content/FreeTier/freetier_topic-Always_Free_Resources.htm

S3 - Oracle AI Database 26ai - VECTOR_DISTANCE: https://docs.oracle.com/en/database/oracle/oracle-database/26/vecse/vector_distance.html

S4 - Oracle - AI Vector Search User Guide / supported clients: <https://docs.oracle.com/en/database/oracle/oracle-database/26/vecse/>

S5 - Oracle - Connect Python Applications with a Wallet (mTLS): <https://docs.oracle.com/en-us/iaas/autonomous-database-serverless/doc/connecting-python-mtls.html>

S6 - Ollama - Linux installation including ARM64: <https://docs.ollama.com/linux>

S7 - Oracle - LangChain Python Integration: <https://docs.oracle.com/en/database/oracle/oracle-database/26/aintg/langchain-oracledb-integration-guide/langchain-python.html>

S8 - Ollama - Qwen2.5 model library: <https://ollama.com/library/qwen2.5>

S9 - Ollama - nomic-embed-text model library: <https://ollama.com/library/nomic-embed-text>

S10 - Ollama - nomic-embed-text model metadata (embedding length 768): <https://ollama.com/library/nomic-embed-text/blobs/970aa74c0a90>

S11 - python-oracledb - Autonomous Database connection handling: https://python-oracledb.readthedocs.io/en/latest/user_guide/connection_handling.html

S12 - OpenAI - API pricing: <https://developers.openai.com/api/docs/pricing>

S13 - OpenAI Help - API billing is separate from ChatGPT subscriptions: <https://help.openai.com/en/articles/8156019-is-api-usage-included-in-chatgpt-subscriptions-even-if-i-have-a-paid-chatgpt-account>

Companion document to: Secure Agentic AI Banking Demo - OCI Always Free Step-by-Step Runbook